

MOBILE APPLICATION AND DEVELOPMENT • WEEK 3 • THEORY 1

Core Android Components

Activities • Services • Broadcast Receivers • Content Providers

Ekramul Haque Amin • Lecturer, AI & Data Science • Dhaka International University

TODAY'S ROADMAP

What We Will Cover

- 1 Why Android apps are built from components**
- 2 Activities and the Activity lifecycle**
- 3 Services: background work without a UI**
- 4 Broadcast Receivers: reacting to system events**
- 5 Content Providers: sharing structured data**

WHERE WE LEFT OFF

Recap: Week 2

Theory 1

Built Hello World and ran it on an AVD

Theory 2

Version control with Git and GitHub

Today

Look inside the app: what is it actually made of?

Every app you have built so far already contains at least one component: MainActivity.

A DIFFERENT APP MODEL

Apps Are Built from Components

An Android app is not one monolithic program. It is a set of loosely-coupled building blocks called application components. The system can start, stop or destroy any of them independently, and can even hand control to another app.

Independent

Each component has its own lifecycle

Reusable

Other apps can start your components, and you can start theirs

System-managed

Android decides when to create and destroy them

THE FOUR BUILDING BLOCKS

Android's Core Components

Activity

A single screen with a UI. Example: a login screen.

Service

Background work with no UI. Example: playing music while you use another app.

Broadcast Receiver

Listens for system-wide or app events. Example: battery low, network changed.

Content Provider

Manages and shares structured app data. Example: exposing contacts to other apps.

HOW COMPONENTS START

Intents Activate Components

Activities, Services and Broadcast Receivers are started by an Intent, a message describing what should happen. Content Providers are reached through a ContentResolver instead.

```
startActivity(new Intent(this, DetailActivity.class));  
startService(new Intent(this, MyService.class));  
sendBroadcast(new Intent("com.example.MY_EVENT"));  
getContentResolver().query(uri, null, null, null, null);
```

Every component must also be declared in AndroidManifest.xml, or the system does not know it exists.

PART 1 – ACTIVITIES

What Is an Activity?

An Activity is a single, focused screen the user can interact with: composing an email, viewing contacts, taking a photo. Each screen is usually its own Activity class, and activities work together to form one user experience.

One screen, one class

LoginActivity, ProfileActivity, SettingsActivity

Largely independent

Each Activity has its own layout and lifecycle

Launched by an Intent

The launcher Activity is declared in the manifest

CREATED, VISIBLE, INTERACTIVE

The Activity Lifecycle

```
onCreate() → onStart() → onResume()
                        |
                    Activity is RUNNING
                        |
onDestroy() ← onStop() ← onPause()
                |
            onRestart() → onStart() (user comes back)
```

The system moves an Activity between these states as the user navigates, rotates the device, takes a call or switches apps.

WHAT EACH CALLBACK IS FOR

Lifecycle Callbacks

Callback	Called when	Typical use
<code>onCreate()</code>	Activity is first created	<code>setContentView</code> , view bindings
<code>onStart()</code>	Just before it becomes visible	Start UI resources, animations
<code>onResume()</code>	It starts interacting with the user	Resume camera, sensors, updates
<code>onPause()</code>	Another Activity comes to front	Save unsaved data, pause video
<code>onStop()</code>	No longer visible	Release resources not needed
<code>onDestroy()</code>	Before it is destroyed	Final cleanup

THE EASY ONE TO FORGET

onRestart() and the Back Stack

onRestart() runs in exactly one case: the Activity was stopped (the user navigated away and **onStop()** ran) and is now being started again (the user navigated back).

First launch

onCreate → onStart → onResume

Return after leaving

onRestart → onStart → onResume

Why it matters

Re-acquire anything you released in onStop()

REGISTERING A SCREEN

Declaring an Activity in the Manifest

```
<activity
  android:name=".MainActivity"
  android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
```

MAIN + LAUNCHER marks the Activity that opens when the user taps the app icon.

SEE IT HAPPEN IN LOGCAT

Lifecycle Demo Code (Java)

```
public class MainActivity extends AppCompatActivity {  
    private static final String TAG = "LifecycleDemo";  
    @Override protected void onCreate(Bundle b) {  
        super.onCreate(b);  
        setContentView(R.layout.activity_main);  
        Log.d(TAG, "onCreate()");  
    }  
    @Override protected void onStart() { super.onStart(); Log.d(TAG, "onStart()"); }  
    @Override protected void onResume() { super.onResume(); Log.d(TAG, "onResume()"); }  
    @Override protected void onPause() { super.onPause(); Log.d(TAG, "onPause()"); }  
    @Override protected void onStop() { super.onStop(); Log.d(TAG, "onStop()"); }  
    @Override protected void onDestroy() { super.onDestroy(); Log.d(TAG, "onDestroy()"); }  
}
```

Always call the super method first. Filter Logcat by the tag LifecycleDemo.

PREDICT THE CALLBACKS

What Happens When...

The user...	Callbacks fired
Opens the app	onCreate, onStart, onResume
Presses Home	onPause, onStop
Returns to the app	onRestart, onStart, onResume
Rotates the device	onPause, onStop, onDestroy, then onCreate again
Presses Back	onPause, onStop, onDestroy

Rotation destroys and recreates the Activity, which is why unsaved data can be lost.

PART 2 – SERVICES

What Is a Service?

A Service runs in the background to perform long-running operations, or work for another process, without providing a user interface. It keeps working while the user interacts with a different Activity or even a different app.

Play music

Keeps playing after you leave the app

Fetch data

Downloads over the network without blocking the UI

Not a thread

A Service runs on the main thread unless you create one

THREE FLAVOURS

Types of Services

Started

Begun with `startService()`. Runs on its own until it stops itself or is stopped.

Bound

Begun with `bindService()`. Offers a client-server interface to send requests and get results.

Foreground

User is actively aware of it. Must show a persistent notification. Rarely killed by the system.

Examples: file download (started), a music player controlled by an Activity (bound), navigation or media playback (foreground).

STEP BY STEP

Creating a Service

- 1 Right-click the package: New > Service > Service, e.g. MyService
- 2 Extend Service and override onCreate, onStartCommand, onBind, onDestroy
- 3 Put the background work in onStartCommand() and return a restart policy
- 4 Return an IBinder from onBind() for bound services, or null if only started
- 5 Declare the <service> tag inside <application> in AndroidManifest.xml
- 6 Start with startService() and stop with stopService() or stopSelf()
- 7 Trigger it from two buttons and watch Logcat for each callback

A BASIC STARTED SERVICE (JAVA)

MyService

```
public class MyService extends Service {  
    @Override public void onCreate() {  
        super.onCreate(); // once, when first created  
    }  
    @Override public int onStartCommand(Intent i, int flags, int startId) {  
        // do background work (NOT on the main thread)  
        return START_STICKY;  
    }  
    @Nullable @Override public IBinder onBind(Intent i) {  
        return null; // not a bound service  
    }  
    @Override public void onDestroy() { super.onDestroy(); }  
}
```

IF THE SYSTEM KILLS IT

onStartCommand() Return Values

START_STICKY

Recreate the service, but with a null Intent

START_NOT_STICKY

Do not recreate unless there are pending start requests

START_REDELIVER_INTENT

Recreate and redeliver the last Intent

Pick by asking: if Android kills this service to free memory, should the work resume, and does it need the original Intent?

REGISTER IT, THEN START IT

Declaring and Starting a Service

```
←!— AndroidManifest.xml, inside <application> →  
<service  
    android:name=".MyService"  
    android:exported="false" />
```

```
// In an Activity  
startService(new Intent(this, MyService.class)); // onStartCommand() runs  
stopService(new Intent(this, MyService.class)); // onDestroy() runs
```

Without the manifest entry the service can never be started.

PART 3 – BROADCAST RECEIVERS

What Is a Broadcast Receiver?

A Broadcast Receiver lets the system and apps send and receive broadcast messages, like publish-subscribe. When an event happens Android broadcasts an Intent, and every registered receiver is notified, even if the app is not currently running.

Publisher

The system or an app sends a broadcast Intent

Subscriber

Your receiver declares interest in that action

Reaction

onReceive() runs and responds

TWO WAYS TO REGISTER

Static vs. Dynamic Receivers

Static (manifest)

Declared in a `<receiver>` tag in `AndroidManifest.xml`. Can react even when the app is not running, with modern background limits.

Dynamic (context)

Registered at runtime with `registerReceiver()` and removed with `unregisterReceiver()`. Active only while the registering component is alive.

Always unregister a dynamic receiver in `onStop()` or `onDestroy()` to avoid leaks.

EVENTS ANDROID ANNOUNCES

Common System Broadcasts

Action	Meaning
BOOT_COMPLETED	The device finished booting
BATTERY_LOW	Battery level is low
CONNECTIVITY_CHANGE	Network connectivity changed

Newer Android versions restrict many implicit broadcasts in the manifest, so some must be registered dynamically.

REACTING TO A BROADCAST (JAVA)

NetworkChangeReceiver

```
public class NetworkChangeReceiver extends BroadcastReceiver {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        Toast.makeText(context, "Network changed",  
            Toast.LENGTH_SHORT).show();  
    }  
}
```

```
// In an Activity: register dynamically  
receiver = new NetworkChangeReceiver();  
registerReceiver(receiver, new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION));  
// later, in onStop()  
unregisterReceiver(receiver);
```

onReceive() must finish quickly. Start a Service for long work.

PART 4 – CONTENT PROVIDERS

What Is a Content Provider?

A Content Provider manages access to a structured set of data (a SQLite database, files or network data) and exposes it through a standard interface so other apps can query, insert, update or delete, subject to permissions.

Contacts

Names, numbers, emails

Calendar

Events and reminders

Media

Photos, audio and video

WHY BOTHER?

Why Use a Content Provider?

- 1 A consistent, permission-controlled way to share data between apps**
- 2 It hides the storage mechanism from the components that consume the data**
- 3 It works with content: URIs used by ListView/RecyclerView adapters and file pickers**

If data is only used inside your own app, you do not need a Content Provider. Use SQLite or Room directly.

ADDRESSING DATA

Content URIs

`content://<authority>/<path>/<optional id>`

`content://com.android.contacts/contacts/5`

content://
Scheme: this is provider data

com.android.contacts
Authority: which provider

contacts/5
Path and id: which row

READING THE CONTACTS PROVIDER (JAVA)

Querying a Content Provider

```
Cursor cursor = getContentResolver().query(
    ContactsContract.Contacts.CONTENT_URI,
    null, null, null, null);
if (cursor != null) {
    while (cursor.moveToNext()) {
        String name = cursor.getString(cursor.getColumnIndexOrThrow(
            ContactsContract.Contacts.DISPLAY_NAME));
    }
    cursor.close();
}
```

Reading contacts needs the `READ_CONTACTS` permission, requested at runtime on modern Android.

SIDE BY SIDE

Choosing the Right Component

Need to...	Use	Started by
Show a screen	Activity	Intent
Work with no UI	Service	Intent
React to an event	Broadcast Receiver	Broadcast Intent
Share app data	Content Provider	ContentResolver

LEARN FROM THESE NOW

Common Mistakes

Forgetting the manifest

Component not declared, so it never starts

Heavy work in onReceive

The receiver gets killed after a few seconds

Blocking the main thread

Services run on the UI thread by default

Not unregistering

Dynamic receivers leak the Activity

IN-CLASS EXERCISE, 15 MINUTES

Watch the Lifecycle

- 1 Add the lifecycle Log.d() calls to your MainActivity**
- 2 Run on the AVD and filter Logcat by LifecycleDemo**
- 3 Press Home, return, then press Back: note the callback order**
- 4 Rotate the emulator and observe what is destroyed and recreated**
- 5 Write down which callback you would use to save a draft message**

BEFORE YOU LEAVE TODAY

Recap Checklist

- 1 Can you name the four core components and what each does?
- 2 Can you list the Activity lifecycle callbacks in order?
- 3 Do you know the difference between started, bound and foreground Services?
- 4 Can you explain static vs. dynamic Broadcast Receivers?
- 5 Do you know what a content URI looks like?

WHERE WE GO FROM HERE

Next Steps

This week: Lab

Components walkthrough demo: lifecycle, a started Service and a receiver

Next theory

Android Project Structure and AndroidManifest.xml in depth

Before the lab, make sure your Hello World project still builds and runs.

Questions?

Activity

Service

Receiver

Provider

Ekramul Haque Amin

Lecturer, AI & Data Science · Mobile Application and Development (0613-409)